



Algoritma & Struktur Data

Week 2 - Enumeration, Data Structure,
Union & File Processing



Last Week Review

- Pointer
 - Variable which simply store an address of other variable.
- Array
 - A Collection of homogeneous data which uniform in size and data type.
 - May be implemented as multi dimensional array which have several usage such as
 - Cartesian Map representation (X,Y,Z) (3 Dimensional array)
 - Matrix Representation (2 Dimensional Array)
- Pointer $\Leftrightarrow$ array relationship.
 - It is possible to access other address block of the array as long as the initial address was defined and stored in the pointer variable.

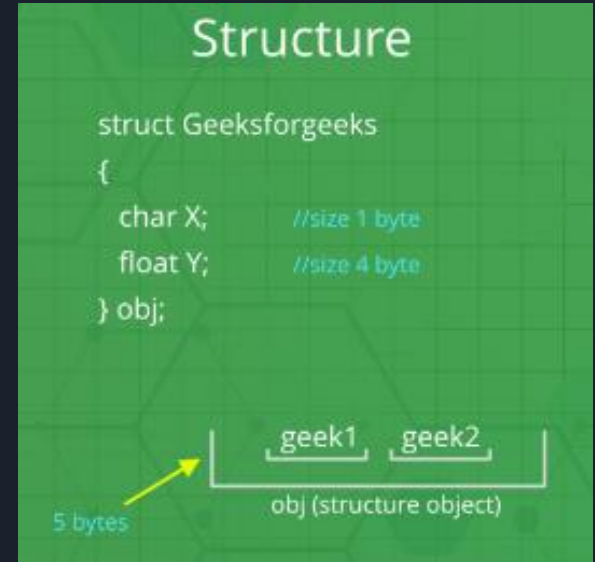


Outline

- Data Structure
 - What
 - How
 - Why
- Enumeration
 - What
 - How
 - Why
- Union
 - What
 - How
 - Why
- File processing
 - What
 - How
 - fopen & fclose (opening the file)
 - fscanf (read input from file)
 - fprintf (write output to the file)
- Discussion Session

Data structure (What)

- User defined Data Type
- A composite data type consist of one or more value with different data type.
- Used to represent complex custom data.
 - Game Character (HP(int), MP(int), Name(Char))
 - Student(Score(int), Name(int), Gender(int))



Data structure (How)

```
#include <stdio.h>

// Declaration of union is same as structures
/*
struct <struct_name>{
    data_type member1;
    data_type member2;
};
*/

struct student{
    char name[100];
    int score;
};

int main()
{
    //initialization struct <struct_name> var_name
    struct student new_student;
    // input the struct members by using . symbol to access its member
    scanf("%s %d",new_student.name,&new_student.score);
    // output the struct members by using . symbol to access its member
    printf("Nama murid: %s\n",new_student.name);
    printf("Nilai murid: %d\n",new_student.score);
    return 0;
}
```

Constraint:

- Any data type is allowed
- Pointer is allowed
- Size of variable is a cumulative size of its member
 - Char + Int = 5
 - Int + Int + Int = 12
 - Char + Char + Int = 6
- Input and output follow general rule of C.
- Accessing member of struct was done through '.'(dot) symbol



Data Structure (Why)

- It is useful to construct composite data with various attribute.
- Example:
 - Each student have a score.
 - without struct
 - `Int score[100]; // store 100 student value`
 - `Char name[100][100] //store 100 student name`
 - Hard to keep track and prone to error.
 - With struct
 - `Student data[100]; ← include score and name`
 - Easier to access and less prone to an error
 - Data was clustered into one set of variable.

Enumeration (What)

- User defined Data Type
- Used to represent the names on integral constants such as
 - Days in Week:
 - Monday(1), Tuesday(2), ..., Sunday(7)
 - Months in Year
 - January(1), February(2).. And so on
 - Error Code
 - NOT FOUND (404)
 - Server ERROR (500)

```
// An example program to demonstrate working
// of enum in C
#include<stdio.h>

enum week{Mon, Tue, Wed, Thur, Fri, Sat, Sun};
enum year{Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec};

int main()
{
    enum week day;
    enum year month;
    day = Wed;
    month = Oct;
    printf("%d\n", day);
    printf("%d", month);
    return 0;
}
```

2
9

Enumeration (How)

```
//basic usage
//enum <enum_name>{data,data2,data3,data4}
enum week{Mon, Tue, Wed, Thur, Fri, Sat, Sun};

//this enum content would be :
// Jan=1, Feb=2, Mar=3,... Dec=12
enum year{Jan=1, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec};

//this enum content would be :
// Jan=0, Feb=1, Mar=2,... Dec=11
//enum year{Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec};

//this enum content would be :
// Jan=0, Feb=2, Mar=3,... Dec=12
//enum year{Jan, Feb=2, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec};
/*
    Constraint:
    Basically it will automatically assign the number in
    increasing order if it's not defined.

    However it is also possible to provide custom value if the
    value does not need to be automatically assigned

    if after defined value, the next value is not defined, it will
    resume from last value.
*/
```

Constraint:

- Basically it will automatically assign the number in increasing order if it's not defined.
- However it is also possible to provide custom value if the value does not need to be automatically assigned
- if after defined value, the next value is not defined, it will resume from last value.
- Every enum constant naming follow the rules of common variable naming.



Enumeration (Why)

- Enumeration used to represent numerical value
- Enable readable constant
- Enumeration improve readability in the code we write.
- Improve productivity during team collab.

Union (What)

- User defined Data Type
- A composite data type consist of one or more value with different data type.
- It shares a same memory address
- If any of its member modified. It will affect the rest of the member.

Structure

```
struct Geeksforgeeks
{
    char X;      //size 1 byte
    float Y;     //size 4 byte
} obj;
```



Unions

```
union Geeksforgeeks
{
    char X;
    float Y;
} obj;
```



Union (How)

```
#include <stdio.h>

// Declaration of union is same as structures
/*
union <union_name>{
    data_type member1;
    data_type member2;
};
*/

union joint_value{
    int number1;
    int number1_replica;
};

int main()
{
    union joint_value data;
    data.number1 = 1;
    //replica will output 1 too.
    printf("%d ",data.number1_replica);
    return 0;
}
```

Constraint:

- The size is defined as the size of largest data type in it's member.
- One modification in it's member will affect the rest of other member.
- Possible to have members with different data type.



Union (Why)

- Union enable us to get different representation of a single value. Which may be useful in several case such as:
 - Mapping decimal and floating value.
 - 28.5 and 28 (for rounding down case)
 - Mapping ascii code to its char counterpart
 - 65 $\rightarrow$ 'A'
 - 66 $\rightarrow$ 'B'
 - Memory optimization on Binary tree construction (Advanced topic, will get back to this in the future lectures)



File Processing (What)

- In programming, data source is important.
- File processing enable us to retrieve the data source from text file.
- In C, file processing is provided by standard library.
- Several option are available for accessing data
 - 'a' = append (add new line to our data, the file does NOT need to exist)
 - 'r' = read data (the file need to exist)
 - 'w' = write data (the file does NOT to exist)
 - Adding + basically allow us to read and write at the same time
 - 'a+' = append (add new line to our data, the file does NOT need to exist)
 - 'r+' = read and write data (the file need to exist)
 - 'w+' = read and write data (the file does NOT to exist)
 - Extra 'x' command allow us to check if file with such name already exists, works only on C 11
 - 'wx' = will return NULL if file with such name already exists.

Fopen, Fclose, Feof (usage & constraint)

Constraint

- Used to open and access the File.
- it will return NULL if the condition provided is invalid. (see previous slide)
- It allow us to read any file.
- It does not automatically parse the content
- Best practice to close the file once you finished using it.

```
// C program to illustrate fopen()

#include <stdio.h>

struct student{
    char name[100];
    int score;
};

int main()
{
    // pointer demo to FILE
    FILE* student_file;
    // Creates a file "student_data.txt"
    // with file access as read mode
    student_file = fopen("student_data.txt", "r");

    struct student student_list[100];

    if(student_file == NULL){
        printf("File Does Not Exist");
    }
    /* content of student_data.txt
    Eka 65
    Lorentzo 90
    William 100
    Noviana 80
    Vero 85
    */
    int counter = 1;
    // loop to extract every lines in the file.
    // feof will return true if it's end of file (empty line)
    while (!feof(student_file)) {
        char buffer[1000];
        fscanf(student_file, "%s\n", buffer);
        printf("Line %d: %s\n", counter++, buffer);
    }

    // closes the file pointed by demo
    fclose(student_file);

    return 0;
}
```



Fscanf (read the file)

fscanf

- Function used to get an input from the data source.
- Exactly like scanf but the input came from file
- It will fetch the data from the text file based on the limiter set on regex.
- Each iteration with '\n' will force the function to read next line.

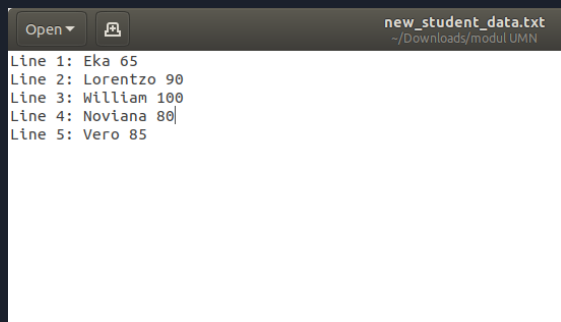
Usage with Regular Expression (regex)

- Allow us to get the string with custom rule set.
- Common usage in file operation:
 - `fscanf(data,"%[^\\n]\\n",scanned_data);`
 - Scan string until one character before '\n' and command file to read another line
 - `fscanf(data,"%[#]##d\\n",student.name,student.score)`
 - Scan string until one character before '#', it used to parse data like:
 - Eka J. Harsono#85
 - `fscanf(data,"%[^$]$$d\\n",student.name,student.score)`
 - Scan string until one character before '#', it used to parse data like:
 - Eka J. Harsono\$85

fprintf

fprintf

- Function used to write an output to file.
- It's exactly like printf but to file



The screenshot shows a text editor window with the title 'new_student_data.txt' and a path '~/.Downloads/modul UMN'. The window contains five lines of text:

```
Line 1: Eka 65
Line 2: Lorentzo 90
Line 3: William 100
Line 4: Noviana 80
Line 5: Vero 85
```

```
// C program to illustrate fopen()

#include <stdio.h>

struct student{
    char name[100];
    int score;
};

int main()
{
    // pointer demo to FILE
    FILE* student_file;
    FILE* new_student_file;
    // Creates a file "student_data.txt"
    // with file access as read mode
    student_file = fopen("student_data.txt", "r");
    new_student_file = fopen("new_student_data.txt", "w");

    struct student student_list[100];

    if(student_file == NULL){
        printf("File Does Not Exist");
    }

    int counter = 1;
    // loop to extract every lines in the file.
    // feof will return true if it's end of file (empty line)
    while (!feof(student_file)) {
        char buffer[1000];
        fscanf(student_file, "%[^\\n]\\n", buffer);
        fprintf(new_student_file, "Line %d: %s\\n", counter++, buffer);
    }

    fclose(student_file);
    fclose(new_student_file);

    return 0;
}
```




References

- <https://www.geeksforgeeks.org/union-c/>



THE END